Musical Tastes: The design and implementation of an AI model to predict human tastes

Miguel Alejandro Chavez Porras Henry Jhonmarcos Osorio Sánchez Juan Andrés Jimenez Palomino
Daniel David Cuellar Cangrejo
Systems Engineering
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia

***Abstract*—The present paper aims to describe and analyze the design and development process of a predictive artificial intelligence (AI) model. The main objective of this model is to determine whether a user is likely to replay a particular song within one month after listening to it for the first time. To address this problem, the model uses historical user behavior data, focusing on interactions with similar tracks. The methodology involves data preprocessing, system analysis and design, and the implementation of feature engineering. This study contributes to understanding user engagement in music streaming platforms and how AI can enhance personalized recommendation systems. The results and conclusions of this research are still in progress.**

***Index Terms*—Music, System Analysis, Artificial intelligence model, user recommendation**

I. Introduction

THE evolution of music consumption has been deeply influenced by the rise of streaming platforms and the development of personalized recommendation systems. As algorithmic models have replaced traditional curators, users now enjoy unprecedented musical diversity and autonomy in their listening choices. Yet, despite these advances, major challenges remain. Recommender systems still struggle to accurately capture individual taste variability and to make reliable predictions when user data is limited or entirely new.

In response to these issues, recent research initiatives have encouraged the creation of models that better understand user behavior and engagement. This work contributes to that effort by designing an AI model that predicts whether a listener will replay a song within a defined period, using historical interaction patterns to enhance music personalization and recommendation accuracy.

Some previous answers tell us about the similarities between this music recommendation problem and the Click value rate and that's because the way those two work and behaves are similar but differing the context, knowing that fact, they built the model and later the solution proposal.

II. Methods and Materials

The dataset provided in this document originates from the KKBox Music Recommendation Challenge. It contains user listening histories, song metadata, and contextual interaction information. Approximately seven million records were available, including categorical and numerical variables such as msno (user ID), song_id, source_system_tab, and song_length. The binary target variable indicates whether a user repeated a given song within thirty days after the first observed listening event.

Before training, the data was cleaned to handle missing or inconsistent values. Numerical values were filled with their median, and missing text values were replaced by a special label. Some new features were created to describe user activity, such as the number of songs a user listened to, how many were repeated, and when the listening happened. Song popularity and artist frequency were also included.

A random forest model was selected to predict whether a user would repeat a song. This algorithm builds many decision trees and combines their results to make a final prediction. It was chosen because it is simple, reliable, and works well with mixed numerical and categorical data. The model uses the features from the previous step to learn patterns between user behavior and song repetition.

The dataset was divided into training and testing parts, using most of the data for training and the rest for evaluation. The model was trained using standard parameters, and the number of trees was adjusted to improve accuracy. The main evaluation metric was the Area Under the ROC Curve, which measures how well the model distinguishes between repeated and non-repeated songs.

System Design and Architecture

To address the challenges of data sensitivity and chaotic user interactions, the proposed architecture follows a modular and layered design. It is divided into five core components: Data Management, Feature Processing, Model Core, Prediction and Feedback, and Monitoring and Control. This design enables each module to operate independently while maintaining coordinated data flow through defined interfaces.

The Data Management Layer is responsible for ingestion, validation, and cleaning of all datasets. It handles inconsistencies in user ages, missing ISRC codes, and duplicated records, ensuring that downstream processes receive reliable inputs. This step minimizes the impact of random or erroneous data on the predictive model.

The Feature Processing Layer transforms validated data into analytical variables used for model training. It applies encoding and normalization routines and generates aggregated metrics representing user habits and song characteristics. These aggregated features reduce the influence of isolated anomalies

or abrupt behavioral changes, increasing the robustness of the learning process.

At the center of the architecture, the Model Core employs a Decision Tree Classifier built with Scikit-learn. This algorithm was chosen for its interpretability and tolerance to nonlinear relationships without requiring complex preprocessing. Training is conducted using time-aware validation to prevent data leakage and respect the temporal structure of the listening events. Each trained model is stored with its parameters and metrics to ensure version control and reproducibility.

The Prediction and Feedback Layer exposes the trained model through a lightweight Flask API. It receives incoming user-song-context tuples and returns the Predicted Relisten Probability, which estimates the likelihood of a user replaying a song. Actual outcomes (target 1/0) are logged to create a feedback dataset that supports model evaluation and retraining, enabling the system to adapt to evolving trends.

The Monitoring and Control Layer oversees the entire architecture to maintain stability under chaotic conditions. Custom Python scripts monitor performance metrics such as AUC and data drift by comparing new input distributions with training data. When significant degradation or drift is detected, the system triggers a retraining cycle or issues calibration updates to the Feature Processing Layer. This closed feedback mechanism prevents long-term instability and supports continuous model refinement.

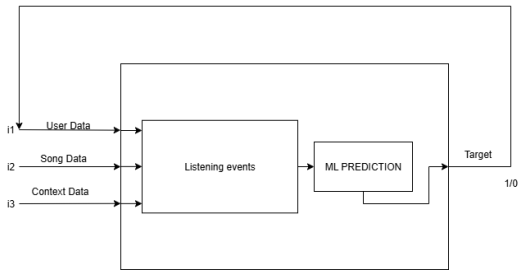


Fig. 1. System Architecture.

Implementation Environment

All components are implemented in Python, chosen for its efficiency, readability, and comprehensive data science libraries. Pandas and NumPy manage data ingestion and transformation, while Scikit-learn supports both preprocessing and model training. Matplotlib provides basic visualization for drift analysis and model performance trends. The system follows a modular pipeline architecture, allowing each component—data handling, feature processing, modeling, feedback, and monitoring—to be developed and tested independently. This structure simplifies debugging, promotes scalability, and facilitates integration of feedback loops that control chaotic variations in user behavior.

III. Results and Discussion

we see a stable and predictable behavior, strongly dependent on data quality, and the level of preprocessing applied. But Performance only improved once the datasets were cleaned; irrelevant noise was mitigated, and consistent category labels were enforced, enabling the model to learn meaningful patterns. $ROC - AUC = 0.6634, Accuracy \approx 0.63$

Reduced 50,000-row inference validation slice:
$ROC - AUC = 0.7345, Accuracy = 0.7435$

These results confirm that the data-driven ML predictor successfully exercised and validated the data

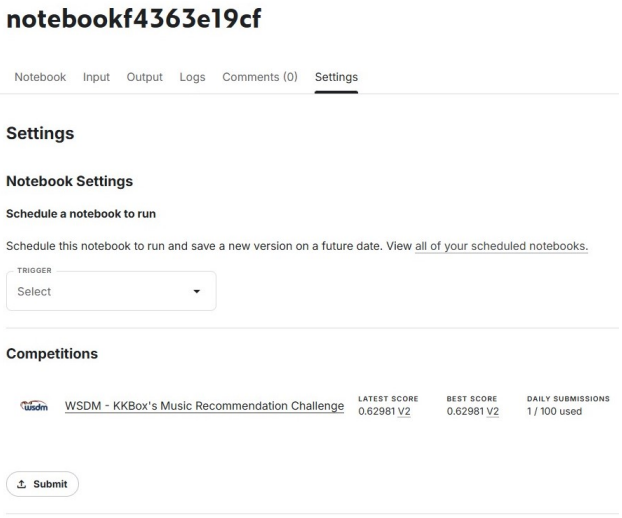


Fig. 2. Kaggle score

The results indicate that the data-driven model is sensitive to noise and uncleaned categories.

The ML model showed sensitivity to the data, exhibiting chaotic-like behavior if not pre-analyzed. The ML predictor, learn from real data and develop a method to categorize.

IV. Conclusions

Dependence on Initial Conditions, the ML model depends on preprocessing data and trust the information cleaned; CA depends on event probabilities so if you change the probabilities or the clean data, the model might go crazy.

The reduced 50k slice is **useful for modular validation but may not be representative of the full data distribution**, which explains the performance of divergence between full and reduced evaluations.

Thorough cleaning, safe imputation, and controlled encoding **substantially improve model performance and reproducibility**. The pipeline reliably produced probabilistic replay predictions with moderate ROC–AUC on the full dataset and improved metrics on a reduced, cleaner slice.

Sensitivity to Noise, like ML performance decreases with noisy categories; CA drastically shifts global behavior with small perturbations.

Both models require a **millimetric calibration** to produce realistic outcomes, making the probabilities and the given and cleared data “realistic”.

References

[1] https://www.kaggle.com/competitions/kkbox-music-recommendation-challenge/overview

[2] https://github.com/EngAndres/ud-public/tree/main/courses/systems-analysis
[3] Systems Analysis and Design, by Alan Dennis, Barbara Haley Wixom, and Roberta M. Roth